

Deep Learning for Natural Language Processing

Assignment 5 — Solutions

Week 5: Transformers

Total: 8 Questions — 10 Marks — (6×1 Mark, 2×2 Marks)

1. An encoder–decoder Transformer is trained with teacher forcing to translate a source sentence of 6 tokens into a target sentence of 5 tokens. Consider the two attention sub-layers of one decoder block while it computes the representation of **target position 3**. Which option correctly states, for both sub-layers, where the queries come from, where the keys and values come from, and how many key positions are visible? **[1 Mark]**
- (A) Masked self-attention: Q from the decoder, K, V from the encoder output; 3 visible positions. Cross-attention: Q, K, V all from the encoder; 6 visible positions.
- (B) Masked self-attention: Q, K, V all from the decoder; 5 visible positions, since teacher forcing makes the whole target available. Cross-attention: Q from the encoder, K, V from the decoder; 6 visible positions.
- (C) Masked self-attention: Q, K, V all from the decoder; 3 visible positions. Cross-attention: Q from the decoder, K, V from the output of the top encoder block; 6 visible positions.
- (D) Masked self-attention: Q, K, V all from the decoder; 3 visible positions. Cross-attention: Q from the decoder, K, V from the top encoder block; 3 visible positions, because the same causal mask is applied there as well.

Answer & Solution

Correct option: (C)

Step 1 — Sub-layer 1: masked (causal) self-attention.

This sub-layer operates *within* the target sequence, so Q, K and V are all linear projections of the decoder’s own representations. The causal mask sets entry $(3, j)$ to $-\infty$ for $j > 3$, so target position 3 sees positions 1, 2, 3 only: **3** visible keys.

Step 2 — Sub-layer 2: encoder–decoder (cross) attention.

The slide states that the output of the top encoder is turned into a set of attention vectors K and V , which every decoder block uses in its encoder–decoder attention layer. The query comes from the decoder’s current state at target position 3. The entire source has already been encoded and contains no “future” relative to the target, so no mask is applied: **all 6** source positions are visible.

Step 3 — Eliminating the distractors.

(A) swaps the two sub-layers: if Q, K, V all came from the encoder there would be no target-side computation at all. (B) makes two errors — it lets position 3 see positions 4, 5 (teacher forcing supplies the correct tokens as *inputs*, but the mask must still hide them, otherwise the next-token label is visible), and it reverses cross-attention. In (D) the cross-attention count is wrong: masking is about not seeing future *target* tokens; the source sentence is fully available.

[Reference: Lecture 24, Slide 87 (How is the decoder different?), Slide 89 (Masked Self-attention for decoder), Slide 90 (Encoder–Decoder Attention), Slide 91 (Decoding)]

2. A single-head **encoder** self-attention layer has $d_{\text{model}} = 3$ and $d_k = d_v = 2$. Two token embeddings are given as row vectors $x_1 = [1, 0, 1]$ and $x_2 = [0, 1, 2]$, and the projections are $q_i = x_i W^Q$, $k_i = x_i W^K$,

$v_i = x_i W^V$ with

$$W^Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad W^K = \begin{bmatrix} 1 & 1 \\ 0 & -1 \\ 0 & 1 \end{bmatrix}, \quad W^V = \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}.$$

Scores are scaled by $\sqrt{d_k}$ and the softmax is taken over the two keys. Using $e^{1.414} \approx 4.113$, $e^0 = 1$ and $\sqrt{2} \approx 1.414$, what is the self-attention output a_1 for token 1? **[2 Marks]**

- (A) [1.609, 1.391]
- (B) [1.762, 1.238]
- (C) [0.392, 2.608]
- (D) [1.000, 2.000]

Answer & Solution

Correct option: (A) $a_1 = [1.609, 1.391]$.

Step 1 — Project the tokens into their three roles.

$$\begin{aligned} q_1 &= x_1 W^Q = [1(1) + 0(0) + 1(1), 1(0) + 0(1) + 1(0)] = [2, 0], \\ k_1 &= x_1 W^K = [1, 2], \quad k_2 = x_2 W^K = [0(1) + 1(0) + 2(0), 0(1) + 1(-1) + 2(1)] = [0, 1], \\ v_1 &= x_1 W^V = [2, 1], \quad v_2 = x_2 W^V = [0, 3]. \end{aligned}$$

Step 2 — Query-key dot products, then scale by $\sqrt{d_k}$.

$$\begin{aligned} q_1 \cdot k_1 &= 2(1) + 0(2) = 2, & q_1 \cdot k_2 &= 2(0) + 0(1) = 0, \\ \frac{2}{\sqrt{2}} &= 1.414, & \frac{0}{\sqrt{2}} &= 0. \end{aligned}$$

Step 3 — Softmax over the key axis.

$$\alpha = \frac{[e^{1.414}, e^0]}{e^{1.414} + e^0} = \frac{[4.113, 1]}{5.113} = [\mathbf{0.804}, \mathbf{0.196}].$$

Step 4 — Weighted sum of the value vectors.

$$a_1 = 0.804[2, 1] + 0.196[0, 3] = [1.609, 0.804 + 0.587] = [\mathbf{1.609}, \mathbf{1.391}].$$

Step 5 — Where the distractors come from.

(B) [1.762, 1.238] omits the $\sqrt{d_k}$ division: unscaled scores 2 and 0 give $\alpha = [0.881, 0.119]$ — note how much sharper the distribution becomes, which is the whole point of scaling. (C) [0.392, 2.608] attaches the weights to the wrong keys (α reversed). (D) [1.000, 2.000] assumes the two scores tie and averages v_1, v_2 , i.e. it ignores the score computation altogether.

[Reference: Lecture 21, Slides 25–26 (An Actual Attention Head: projecting x_i with W^Q, W^K, W^V); Lecture 22, Slides 37–38 (Self-attention: In equations; Calculating the self-attention output)]

3. Two multi-head attention layers are proposed, both with $d_{\text{model}} = 96$, both following the lecture's convention that each head's projections are $d_{\text{model}} \times d_k$ matrices and that W^O has shape $d_{\text{model}} \times h d_v$:

Configuration	heads h	$d_q = d_k = d_v$
A	6	16
B	12	8

Ignoring biases, what is the total number of parameters in all the W^Q, W^K, W^V matrices together with W^O , for A and for B respectively? [1 Marks]

- (A) 27,648 and 27,648
 (B) 36,864 and 36,864
 (C) 36,864 and 73,728
 (D) 13,824 and 11,520

Answer & Solution

Correct option: (B) — both configurations need 36,864 parameters.

Step 1 — Shapes.

Each head projects the model dimension down to d_k , so in A $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{96 \times 16}$, and the heads are concatenated to width $h d_v = 6 \times 16 = 96$, giving $W^O \in \mathbb{R}^{96 \times 96}$. In B the per-head matrices are 96×8 and $h d_v = 12 \times 8 = 96$, so W^O is again 96×96 .

Step 2 — Configuration A.

$$\underbrace{3 \times 6 \times 96 \times 16}_{Q, K, V \text{ over all heads}} = 27,648, \quad \underbrace{96 \times 96}_{W^O} = 9,216, \quad \text{total} = \mathbf{36,864}.$$

Step 3 — Configuration B.

$$3 \times 12 \times 96 \times 8 = 27,648, \quad 96 \times 96 = 9,216, \quad \text{total} = \mathbf{36,864}.$$

Step 4 — Why they are equal.

The projection cost is $3 d_{\text{model}}(h d_k)$ and the output cost is $d_{\text{model}}(h d_v)$; both depend only on the *product* $h d_k$, which equals $d_{\text{model}} = 96$ in each case. Under the constraint $h d_k = d_{\text{model}}$, changing h merely re-partitions a fixed total projection width: it alters how many independent relationships the layer can model, and the sharpness of each head's softmax through $\sqrt{d_k}$, but not the parameter budget.

Step 5 — Where the distractors come from.

(A) omits W^O entirely — forgetting that the concatenated heads must be projected back to d_{model} . (C) assumes that doubling the number of heads doubles the projection parameters, ignoring that d_k is halved at the same time. (D) counts one shared W^Q, W^K, W^V per layer instead of one per head: $3 \times 96 \times 16 + 9216 = 13,824$ and $3 \times 96 \times 8 + 9216 = 11,520$.

[Reference: Lecture 22, Slide 46 (Multi-head Attention Layer: More on dimensions — projection matrices $d \times d_k, W^O : d \times h d_v$); Lecture 23, Slide 76 (Combining from Multiple Heads), Slide 79 (Number of parameters in the encoder?)]

4. A sparse mixture-of-experts layer contains $E = 8$ experts and routes each token independently with top- k routing, $k = 2$. A sequence of 6 tokens passes through the layer. How many token-expert feed-forward computations are performed, how many would be performed if the layer were a Switch layer, and what

fraction of all possible token–expert pairs is evaluated in the first case?

[1 Mark]

- (A) 12; 6; 25%
- (B) 48; 48; 100%
- (C) 12; 12; 25%
- (D) 2; 1; $\approx 4.2\%$

Answer & Solution

Correct option: (A)

Step 1 — Top- k routing.

The MoE layer operates independently on each token in the sequence and the router routes each token independently, so with $k = 2$ the layer performs

$$6 \times 2 = \mathbf{12} \text{ expert evaluations.}$$

Step 2 — Switch layer.

A Switch layer is the $k = 1$ strategy, giving $6 \times 1 = \mathbf{6}$ evaluations.

Step 3 — Fraction of all pairs.

There are $6 \times 8 = 48$ possible token–expert pairs, so $12/48 = \mathbf{25\%}$ are actually computed.

Step 4 — Why this is the point of a sparse MoE.

The parameter count grows with E , while the computation per token is fixed by k . The router still produces all 8 gate values per token (a negligible cost), but only the top- k experts are executed — which is how experts can specialise in specific tokens in specific contexts without every token paying for all of them.

Step 5 — Where the distractors come from.

(B) is the dense case in which every token visits every expert. (C) treats a Switch layer as $k = 2$; the lecture defines it as $k = 1$. (D) gives per-token counts rather than totals for the sequence.

[Reference: Lecture 25, Slide 121 (Sparse Mixture of Experts — the MoE layer operates independently on the tokens; the router routes each token independently), Slide 122 (top- k gating; $k = 1$ is a Switch layer), Slide 123 (What do the experts learn?)]

5. In one attention head a query has dot products 2, 0 and -2 with the three keys of the sequence, and $d_k = 4$. What is the attention distribution over the three keys? [1 Mark]

- (A) [0.867, 0.117, 0.016]
- (B) [0.506, 0.307, 0.186]
- (C) [0.665, 0.245, 0.090]
- (D) [0.333, 0.333, 0.333]

Answer & Solution

Correct option: (C)

Step 1 — Scale by $\sqrt{d_k}$.

$\sqrt{d_k} = \sqrt{4} = 2$, so the scores become $\left[\frac{2}{2}, \frac{0}{2}, \frac{-2}{2}\right] = [1, 0, -1]$.

Step 2 — Softmax over the key axis.

Denominator = $2.718 + 1 + 0.368 = 4.086$, so

$$\alpha = \left[\frac{2.718}{4.086}, \frac{1}{4.086}, \frac{0.368}{4.086} \right] = [0.665, 0.245, 0.090], \quad \sum_i \alpha_i = 1.$$

Step 3 — Where the distractors come from.

(A) skips the scaling entirely and softmaxes $[2, 0, -2]$; the result is visibly sharper, which is what the $\sqrt{d_k}$ division is there to prevent. (B) divides by $d_k = 4$ instead of $\sqrt{d_k} = 2$, giving $[0.5, 0, -0.5]$. (D) assumes that scores summing to zero must produce a uniform distribution — softmax depends on the *differences* between scores, not on their sum.

[Reference: Lecture 22, Slides 53–54 (Training Trick #3: Scaled Dot Product Attention); Lecture 23, Slide 72 (Scaled Dot Product for Attention)]

6. A sequence of $N = 4096$ tokens is processed by a Linformer-style layer, which replaces the $N \times N$ attention-score matrix with an $N \times k$ matrix, $k = 128$. How many entries does each matrix contain, by what factor does the number of score entries fall, and is a 20× reduction achieved? **[1 Mark]**

- (A) 16,777,216 and 524,288; a 16× reduction; the target is not met.
 (B) 8,388,608 and 524,288; a 16× reduction; the target is not met.
 (C) 16,777,216 and 16,384; a 1024× reduction; the target is met.
 (D) 16,777,216 and 524,288; a 32× reduction; the target is met.

Answer & Solution

Correct option: (D)

Step 1 — Count the entries.

$$N^2 = 4096^2 = 16,777,216, \quad Nk = 4096 \times 128 = 524,288.$$

Step 2 — Reduction factor.

$$\frac{N^2}{Nk} = \frac{N}{k} = \frac{4096}{128} = 32 \geq 20,$$

so the requested 20× reduction **is** achieved.

Step 3 — Why this matters.

The ratio depends only on N/k . Because k is held *fixed* while N grows, the cost is $O(Nk)$ rather than $O(N^2)$ — the low-rank projection of the keys and values is what buys the linear behaviour.

Step 4 — Where the distractors come from.

(A) halves the true ratio. (B) additionally halves N^2 , as if only the triangle of the score matrix were stored — but encoder attention is not causal. (C) uses $k^2 = 16,384$, i.e. projects *both* axes down to k ; Linformer projects only the key/value axis, so one dimension stays N .

[Reference: Lecture 25, Slides 117–118 (Linformer — low-rank approximation of the $N \times N$ attention matrix by projecting keys and values, “only require $O(nk)$ time and space”)]

7. A sparse mixture-of-experts layer has $N = 4$ experts. For a token x the router produces logits $h(x) = (1, 2, 0, -1)$, the gate value of expert i is $p_i(x) = e^{h(x)_i} / \sum_{j=1}^N e^{h(x)_j}$, and the layer output is $y = \sum_{i \in \mathcal{T}} p_i(x) E_i(x)$, where $\mathcal{T}$ holds the top- k gates and the gate values are used exactly as produced by the

softmax (no renormalisation). Take $k = 2$, with expert outputs

$$E_1(x) = [1, 0], \quad E_2(x) = [0, 2], \quad E_3(x) = [2, 2], \quad E_4(x) = [-1, 1].$$

What are y and the output the same router would produce in a *Switch* layer?

[2 Marks]

- (A) $y = [0.237, 1.288]$; Switch: $[0, 1.288]$
- (B) $y = [0.269, 1.462]$; Switch: $[0, 2.000]$
- (C) $y = [0.379, 1.494]$; Switch: $[0, 1.288]$
- (D) $y = [1.000, 4.000]$; Switch: $[0, 4.000]$

Answer & Solution

Correct option: (A)

Step 1 — Gate values (softmax over all four experts).

Denominator $= 2.718 + 7.389 + 1 + 0.368 = 11.475$, so

$$p = \left[\frac{2.718}{11.475}, \frac{7.389}{11.475}, \frac{1}{11.475}, \frac{0.368}{11.475} \right] = [0.237, 0.644, 0.087, 0.032].$$

Step 2 — Top- k selection.

The two largest gates are $p_2 = 0.644$ and $p_1 = 0.237$, so $\mathcal{T} = \{2, 1\}$. Experts 3 and 4 are not evaluated for this token at all — that sparsity is the whole point of the layer.

Step 3 — Combine the selected experts.

$$y = 0.644[0, 2] + 0.237[1, 0] = [0 + 0.237, 1.288 + 0] = [0.237, 1.288].$$

Note $0.644 + 0.237 = 0.881 \neq 1$: the lecture's formula sums $p_i(x)E_i(x)$ over the selected set *without* rescaling, so the routed output is deliberately attenuated when the router is unconfident.

Step 4 — Switch layer.

A Switch layer is the $k = 1$ routing strategy, so only the arg-max expert survives: $\mathcal{T} = \{2\}$ and $y = 0.644[0, 2] = [0, 1.288]$.

Step 5 — Where the distractors come from.

(B) renormalises the two surviving gates to $(0.731, 0.269)$, which also forces the Switch gate to 1 and returns E_2 unchanged. (C) is the dense mixture $\sum_{i=1}^4 p_i E_i$ — it uses every expert and so discards the sparsity. (D) uses the raw logits $(2, 1)$ as gate values instead of the softmax probabilities.

[Reference: Lecture 25, Slide 122 (Sparse Mixture of Experts: gate value $p_i(x)$, top- k set $\mathcal{T}$, $y = \sum_{i \in \mathcal{T}} p_i(x)E_i(x)$, “ $k = 1$ routing is later referred to as a Switch layer”), Slide 121 (Switch Transformer encoder block)]

8. A decoder self-attention layer processes a sequence of 7 tokens; entry (i, j) of the 7×7 score matrix is masked whenever $j > i$. How many entries survive the mask, and what fraction of the corresponding encoder count is that? [1 Mark]

- (A) 28 entries; ≈ 0.667
- (B) 28 entries; ≈ 0.571
- (C) 21 entries; ≈ 0.429
- (D) 49 entries; 1.000

Answer & Solution

Correct option: (B)

Step 1 — Count the surviving entries.

Query position i retains exactly i keys (positions 1 through i , the diagonal included since a token attends to itself), so

$$1 + 2 + 3 + 4 + 5 + 6 + 7 = \frac{7 \times 8}{2} = \mathbf{28}.$$

Step 2 — Compare with the encoder.

An encoder applies no mask and uses all $7^2 = 49$ entries, so the decoder computes

$$\frac{28}{49} = \frac{4}{7} \approx \mathbf{0.571}$$

of them; masking removes 21 query–key pairs, one for every future position of every token.

Step 3 — Where the distractors come from.

(A) divides by $42 = 7 \times 6$. (C) counts the strictly lower triangle, i.e. forbids a token from attending to itself — but the diagonal $q_i \cdot k_i$ is retained in the lecture’s mask diagram. (D) assumes the mask is applied only at inference.

[Reference: Lecture 24, Slides 88–89 (Masked Self-attention for decoder — the lecture’s $N \times N$ diagram with $-\infty$ above the diagonal)]